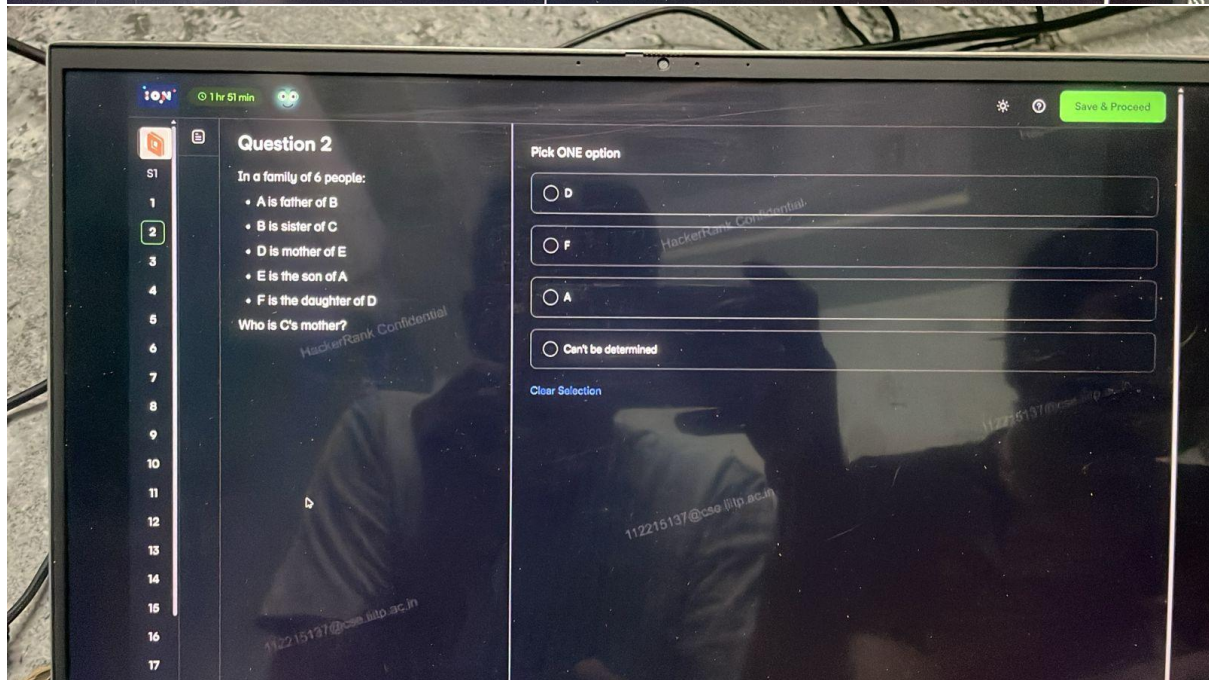
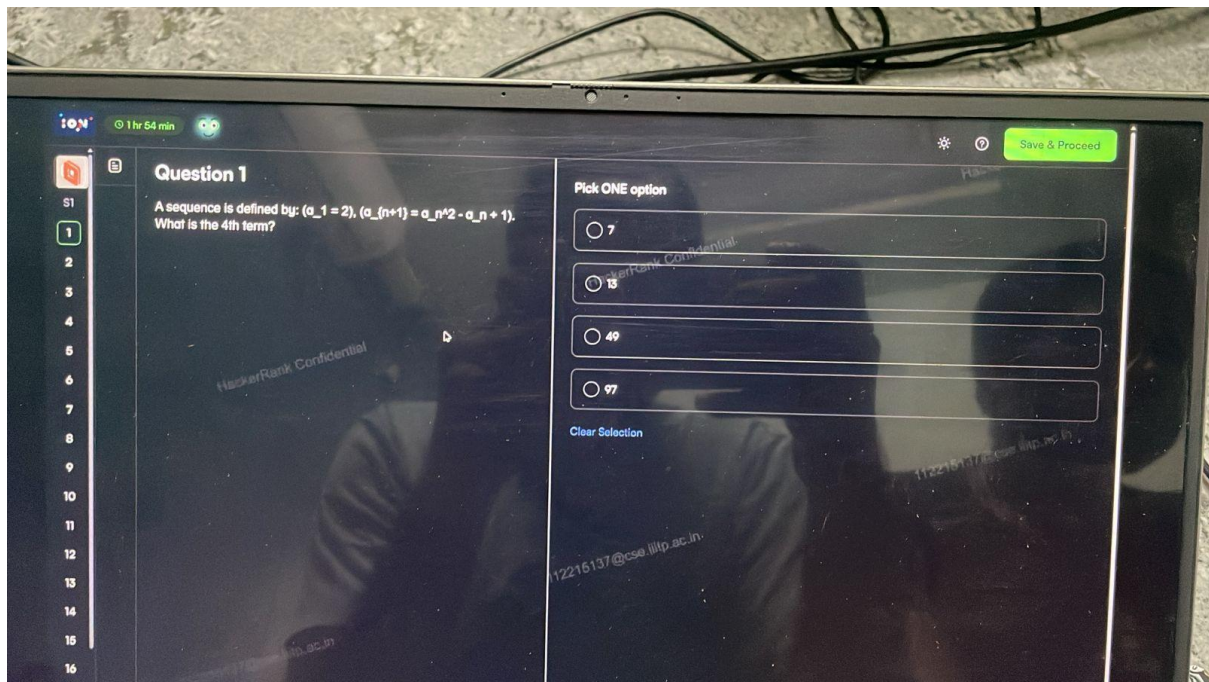
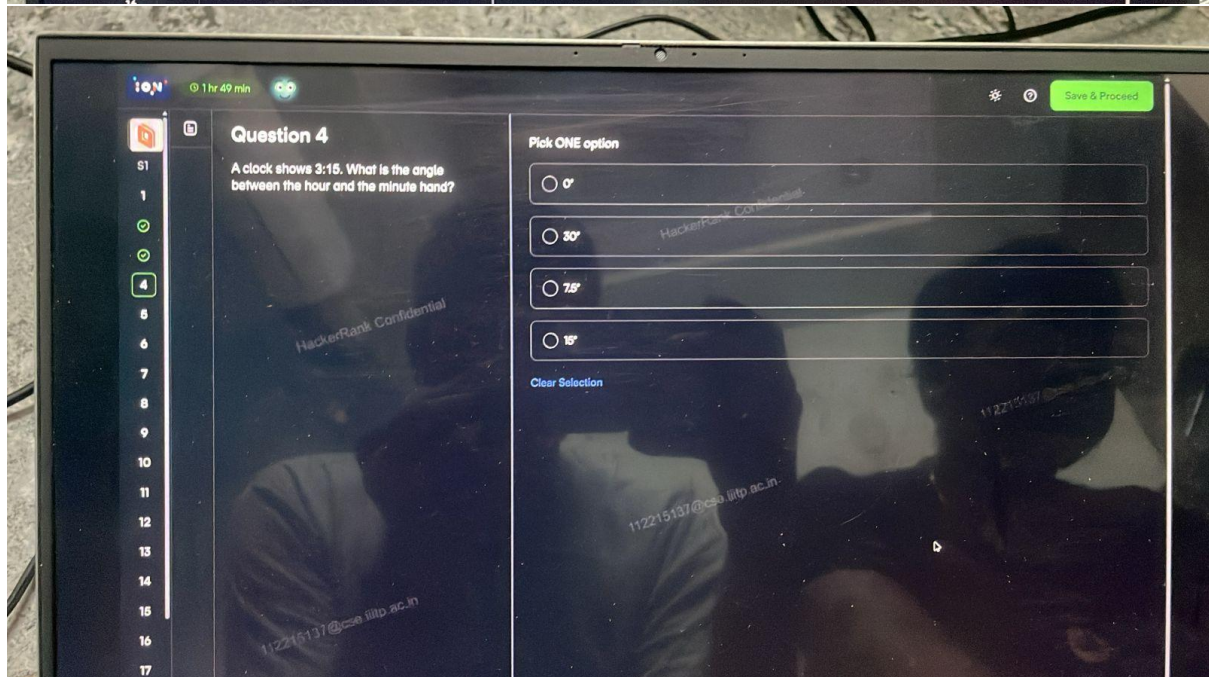
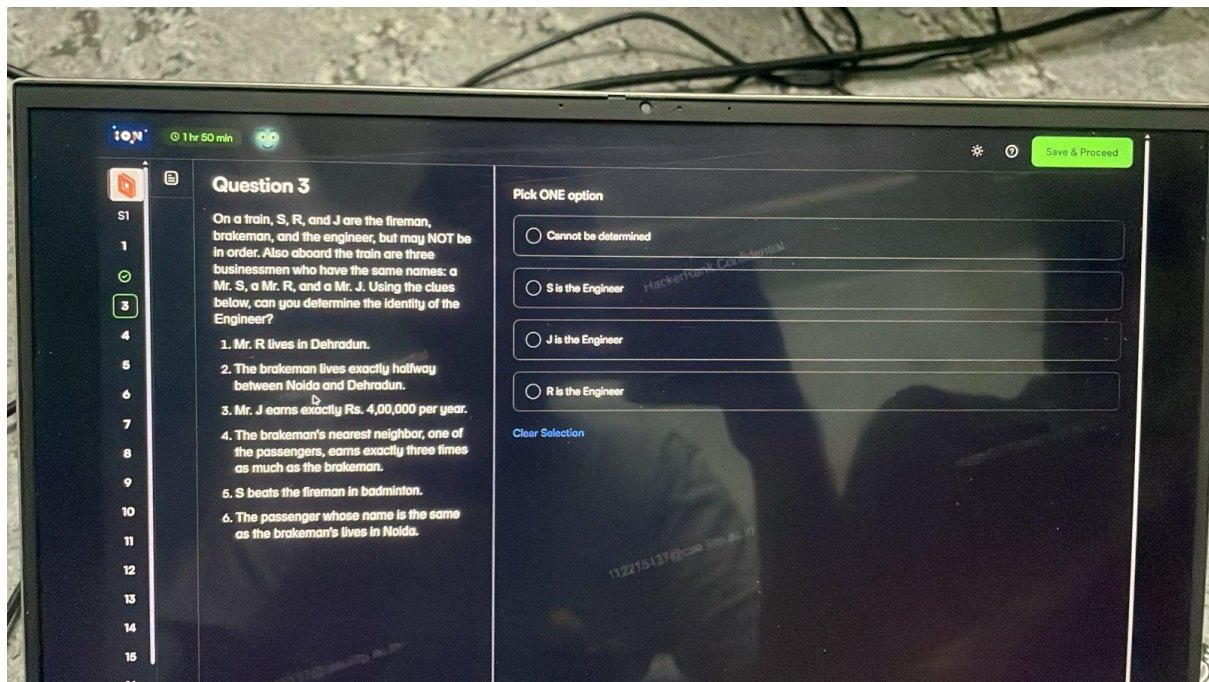
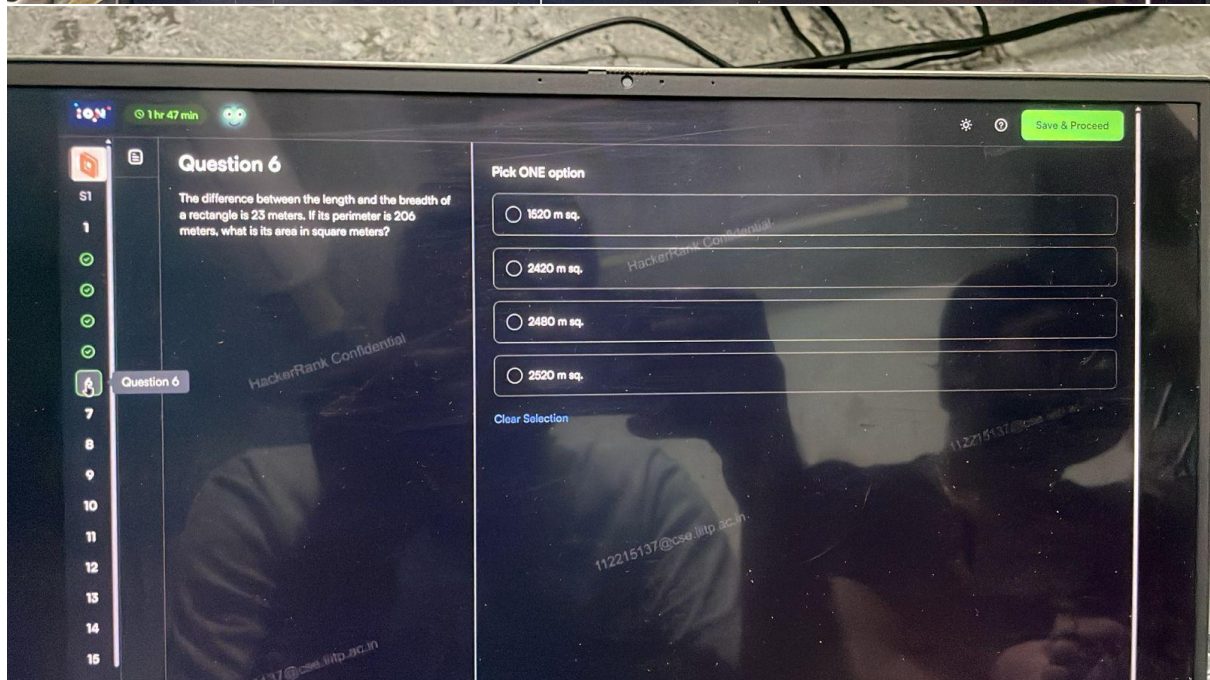
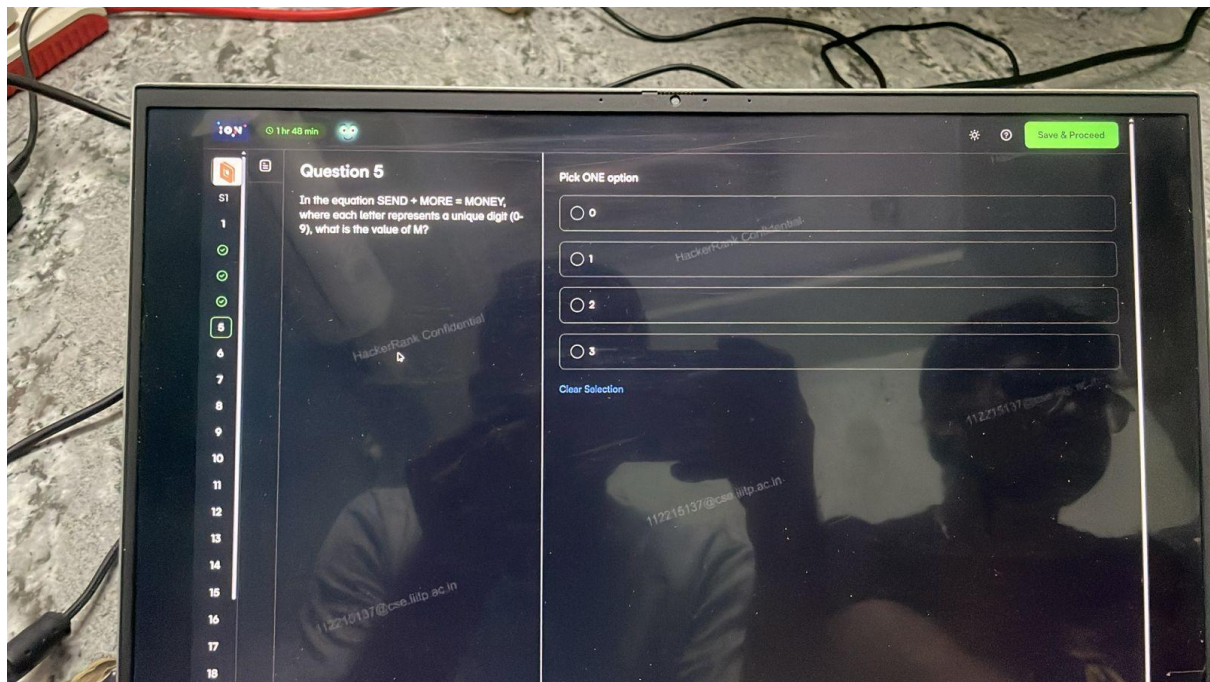
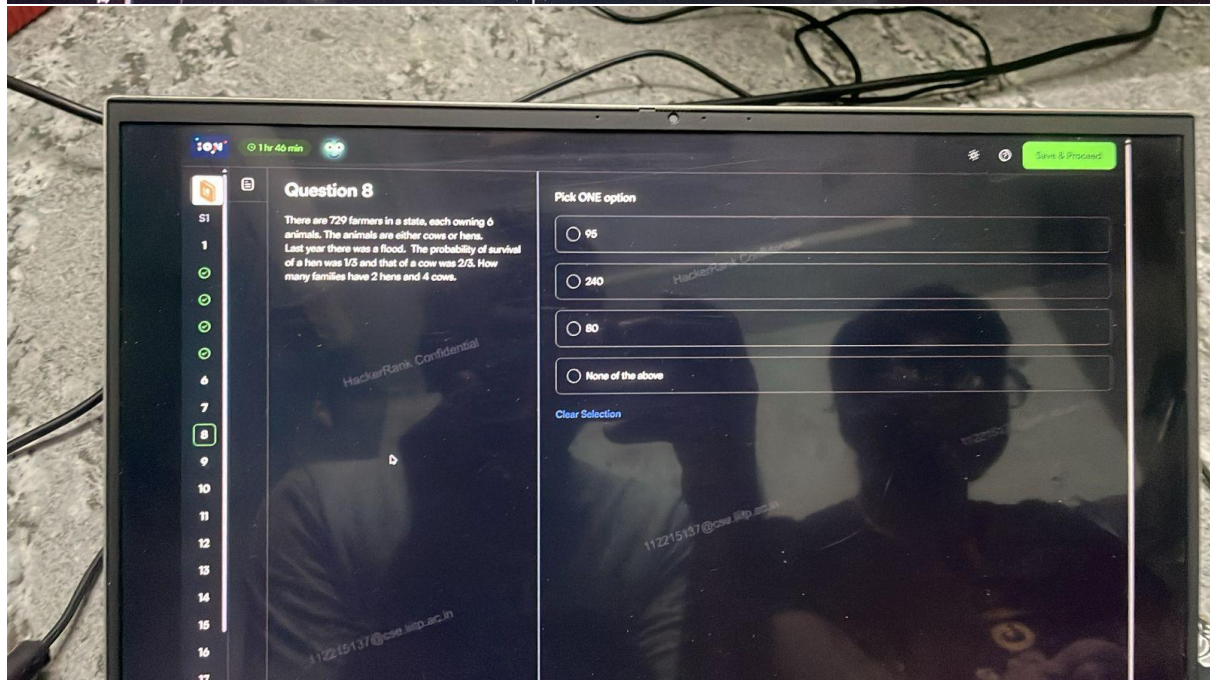
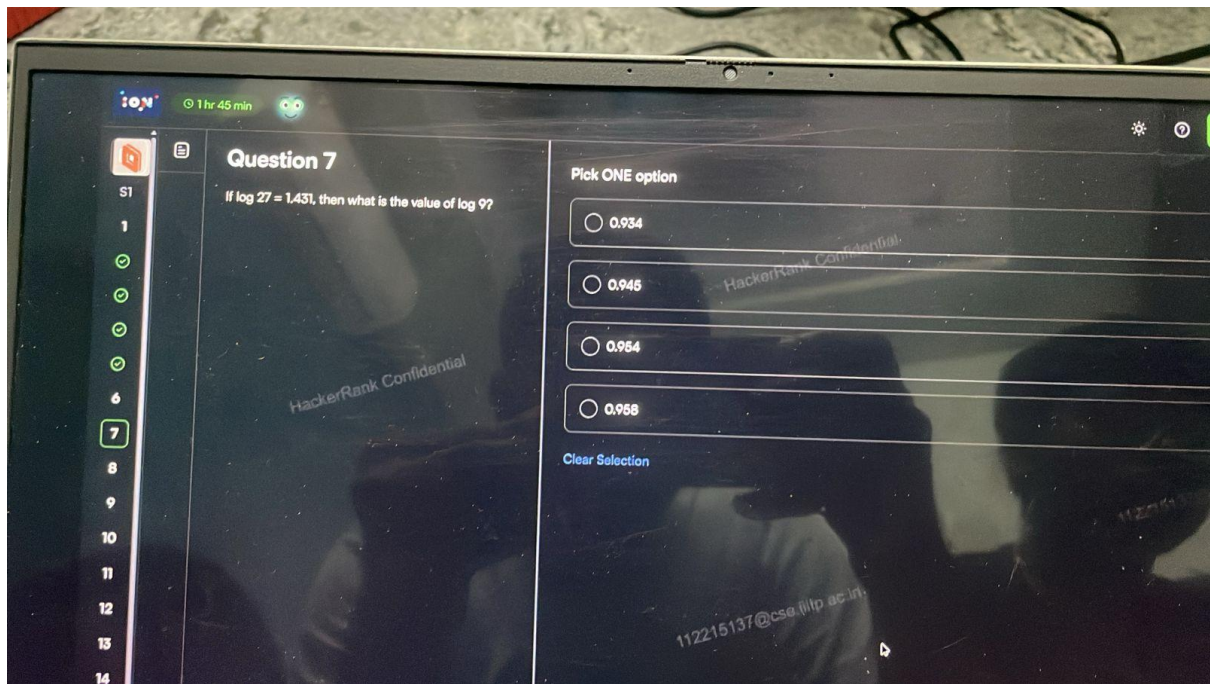
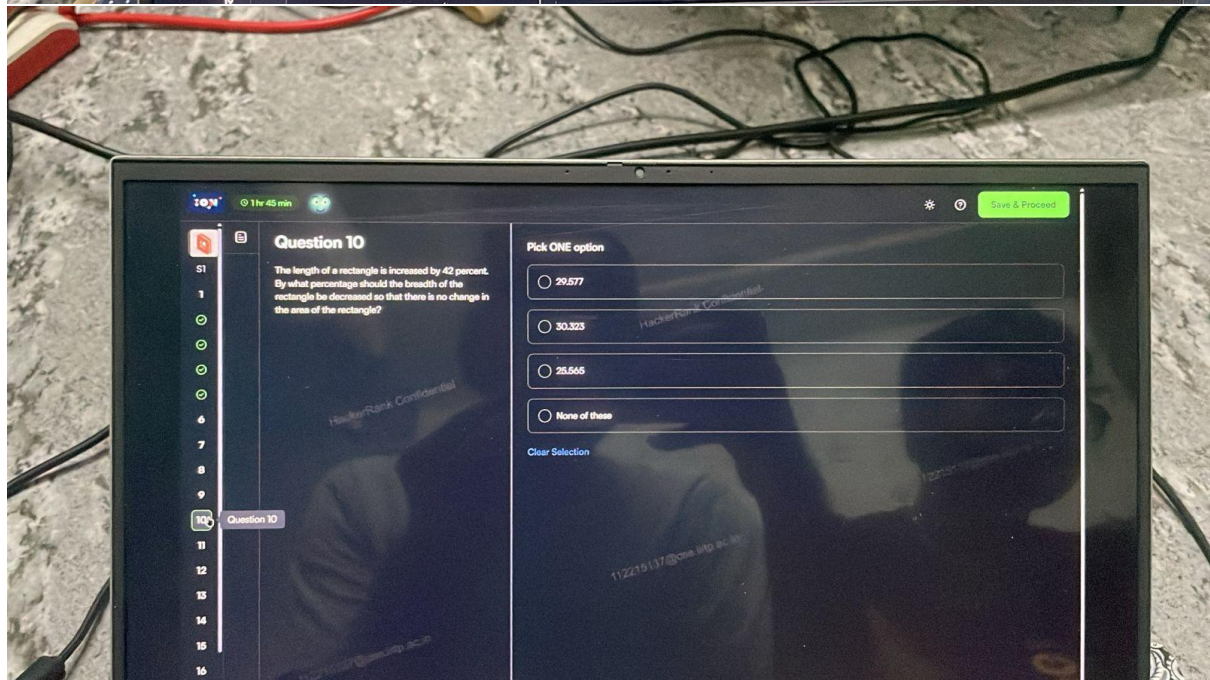
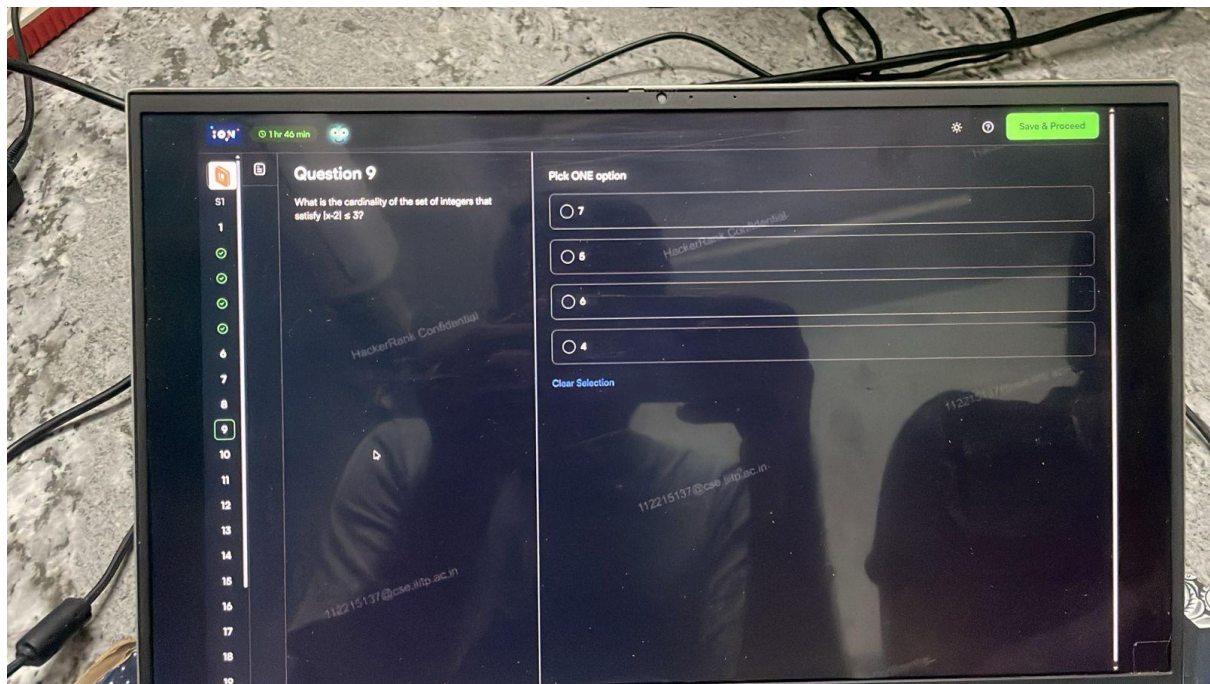


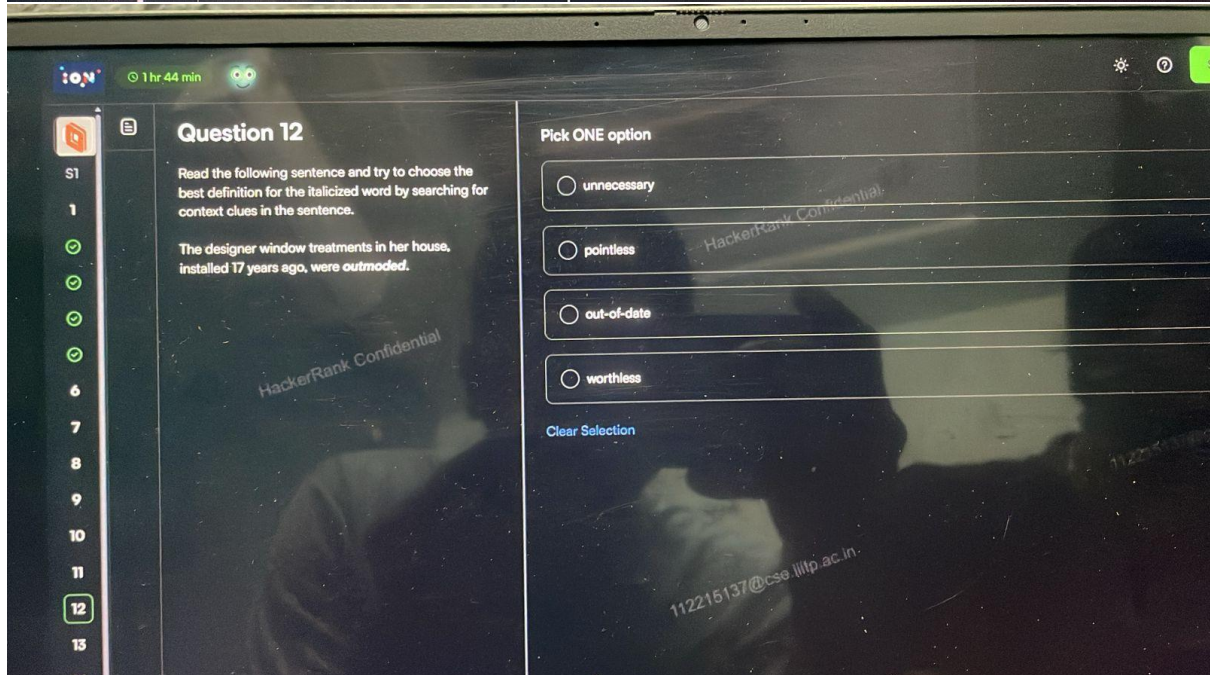
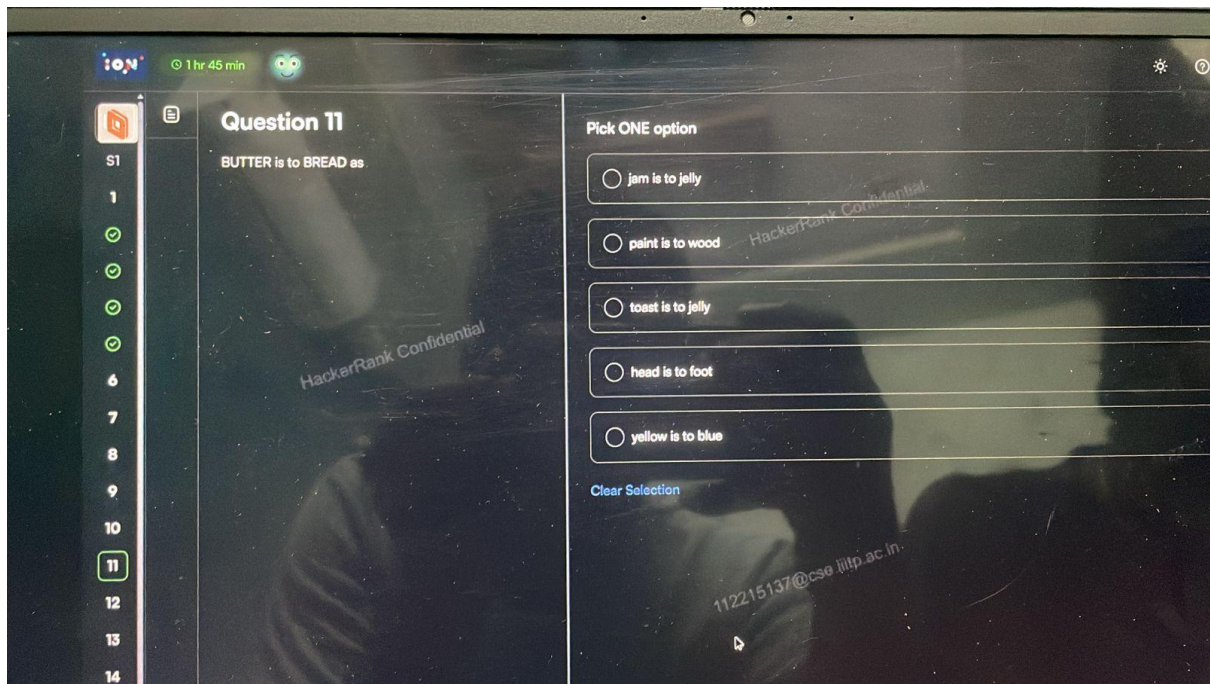


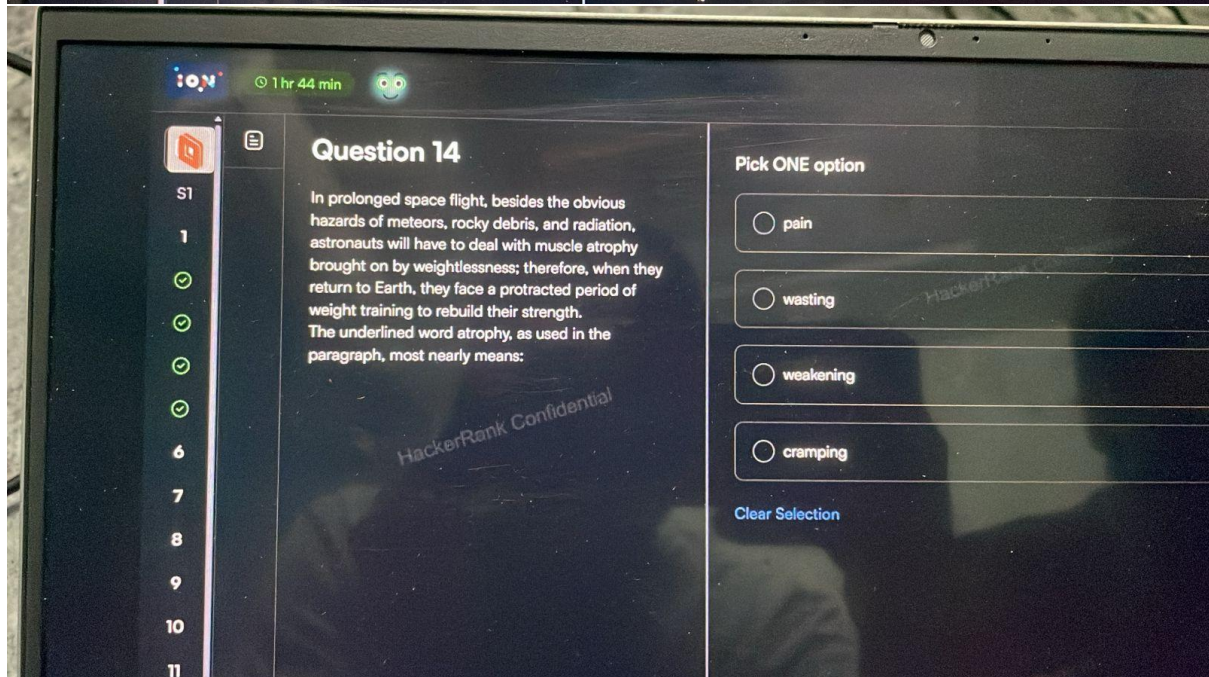
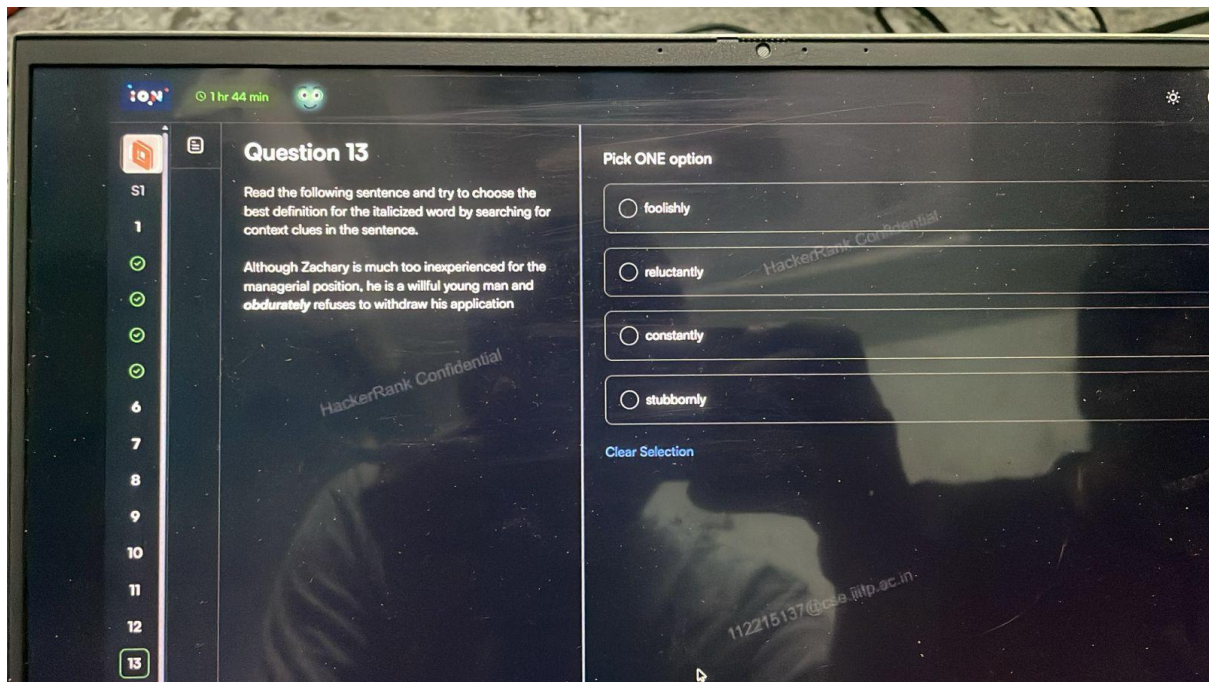


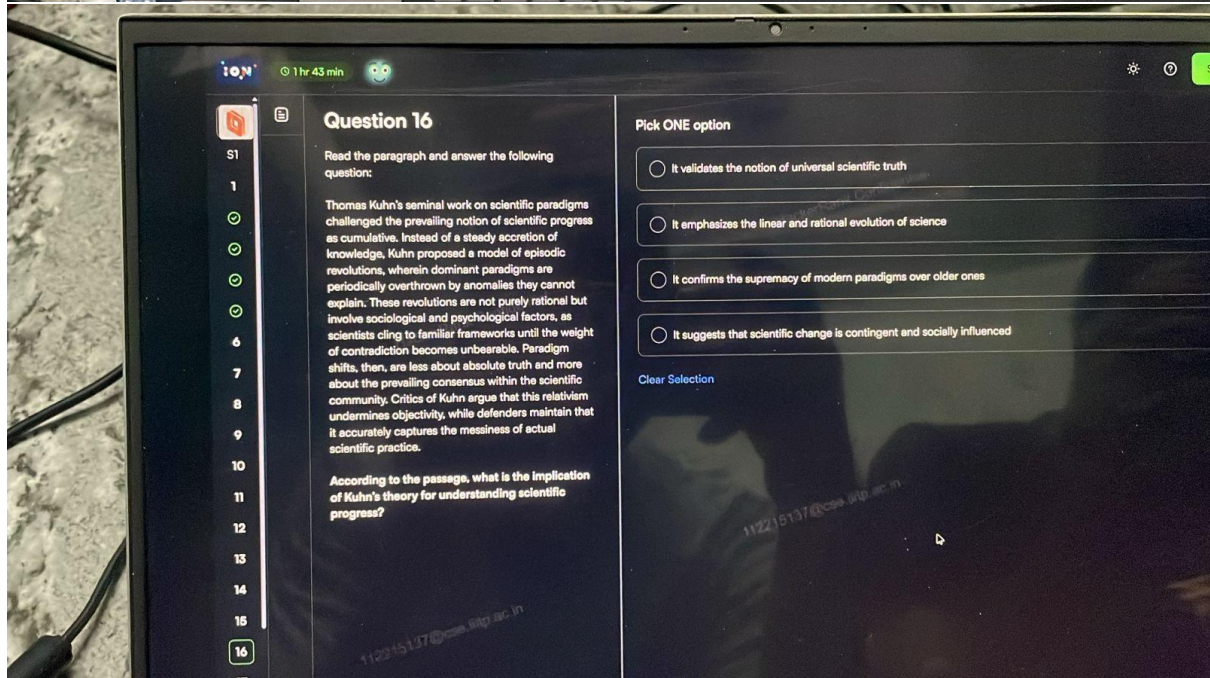
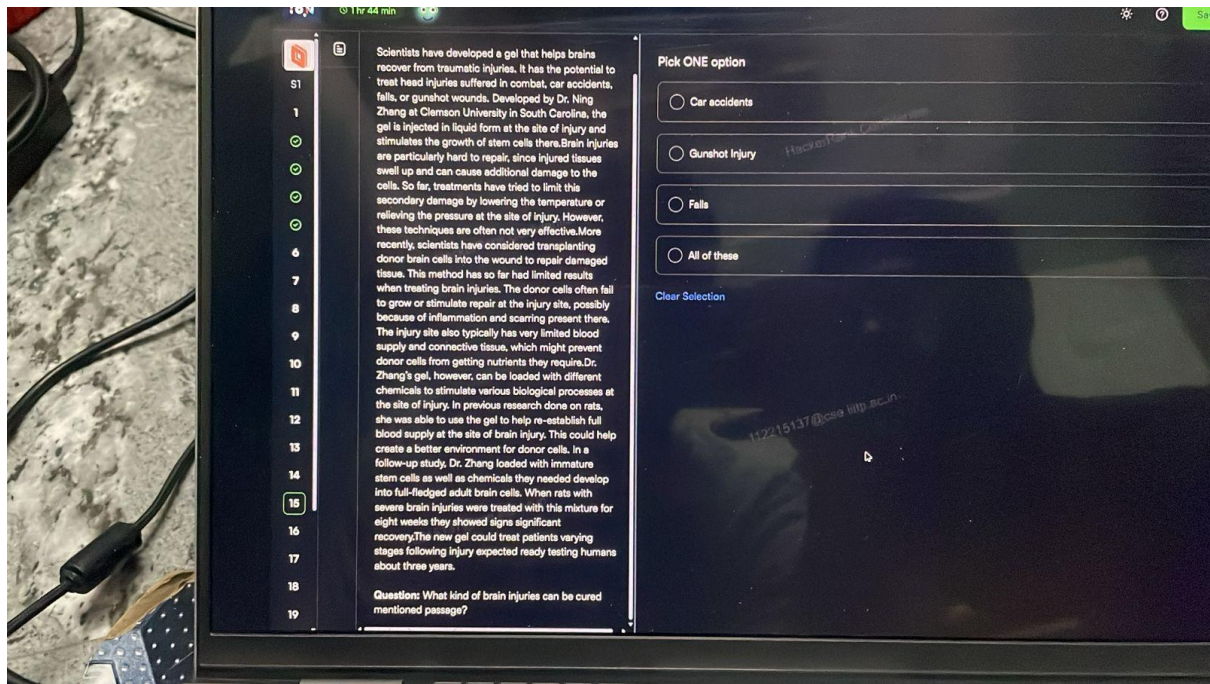


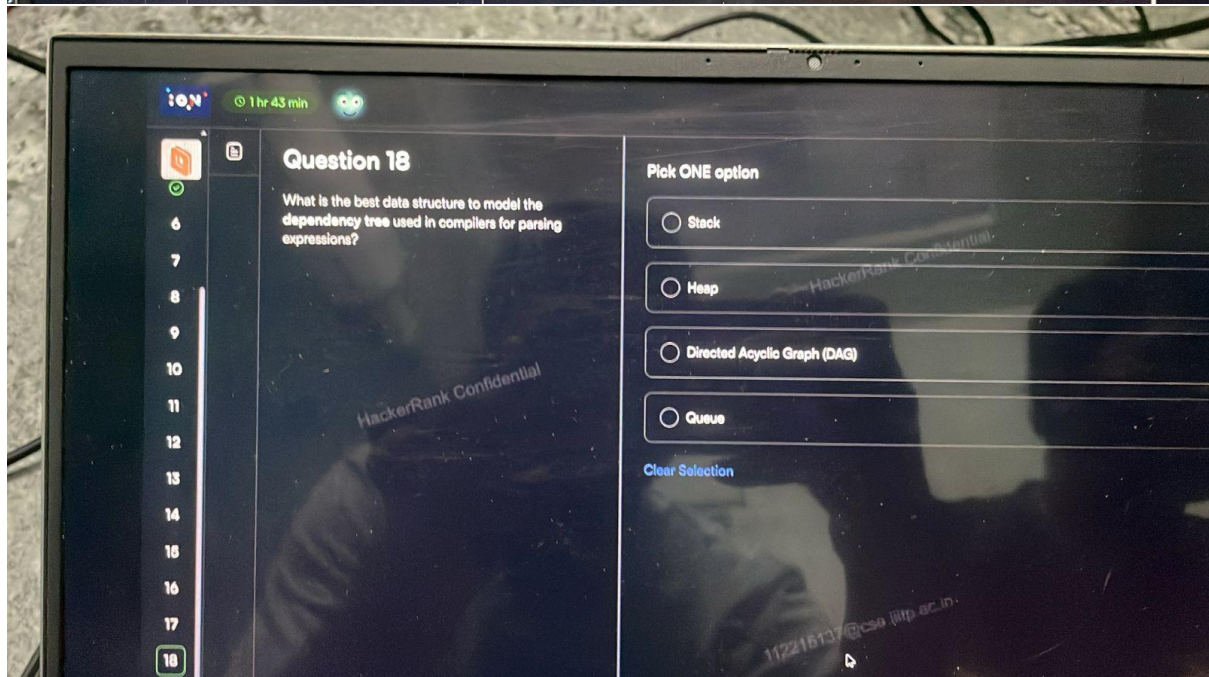
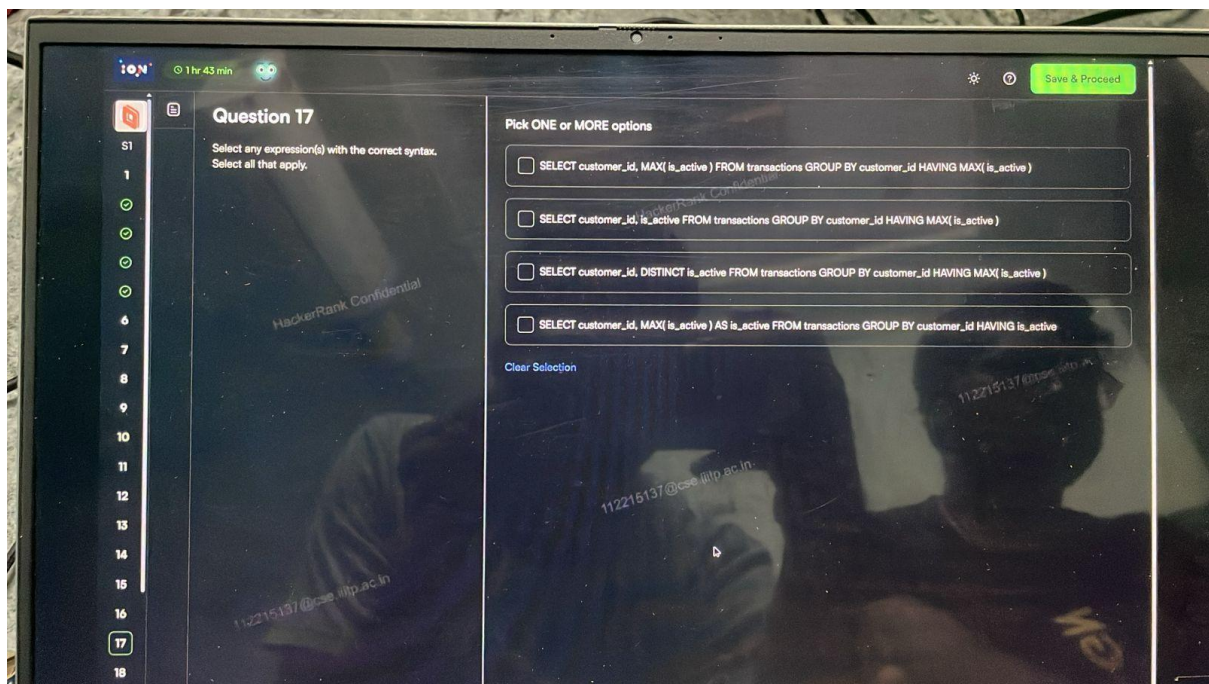


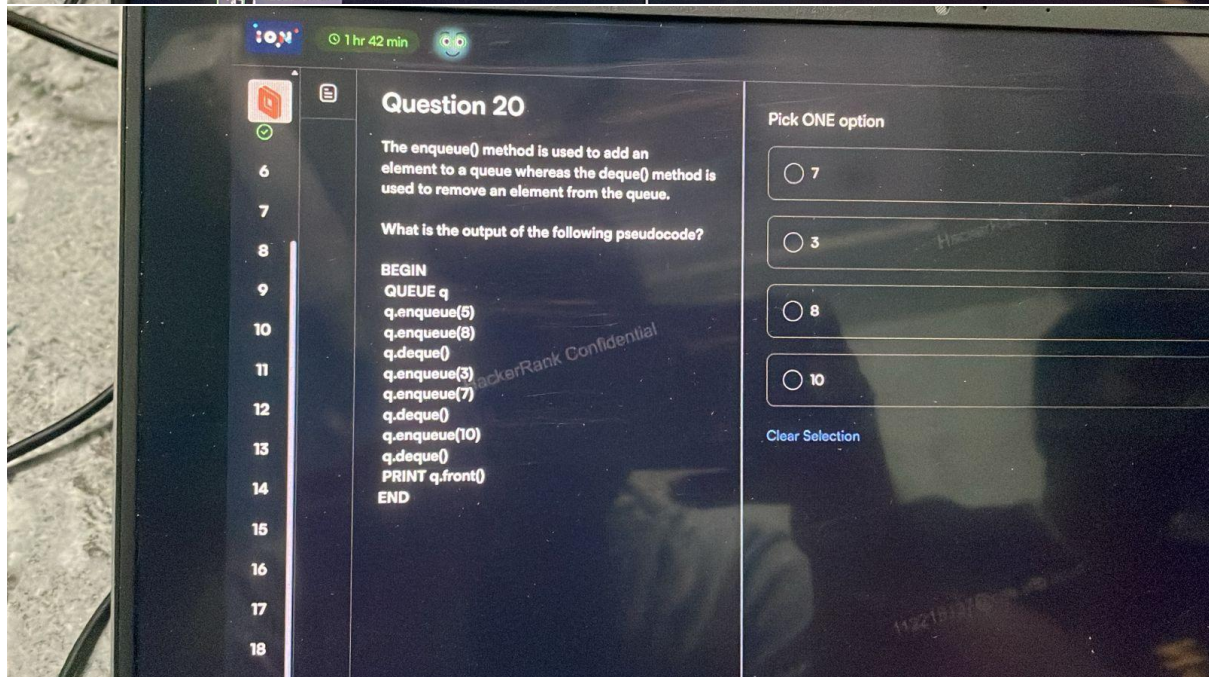
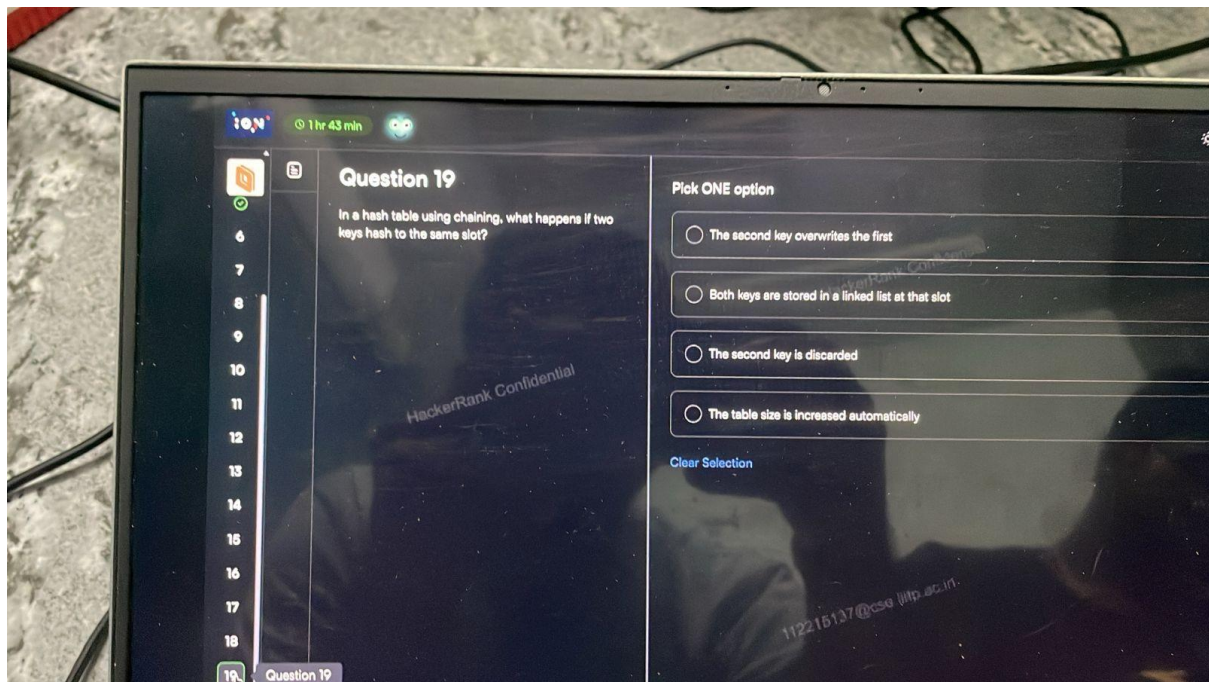












1 hr 42 min

Question 21

Which sorting algorithm does this code implement?
What is its time complexity?

```
void sort(int arr[])
{
    int n = arr.length;
    for (int i = 1; i < n; ++i) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}
```

Pick ONE option

☐ It is the bubble sorting algorithm. Complexity: $O(n^2)$

☐ It is the insertion sort algorithm. Complexity: $O(n^2)$

☐ It is the insertion sort algorithm. Complexity: $O(n \log n)$

☐ It is the quick sort algorithm. Complexity: $O(n^2)$ (worst case)

Clear Selection

1 hr 42 min

Question 22

What is the time complexity of this code?

```
int temp = 0;
for(int i = 1; i <= n; i++) {
    for(int j = 1; j <= n; j+=i) {
        for(int k = 1; k <= n; k+=2) {
            temp++;
        }
    }
}
```

Pick ONE option

☐ $O(n \log(n))$

☐ $O(n \log^2(n))$

☐ $O(n^2 \log(n))$

☐ $O(n^2 \log^2(n))$

Clear Selection


```

vector<string> missingWords(string s, string t) {
    vector<string> result;
    istringstream ss(s), tt(t);
    vector<string> s_words, t_words;

    string word;
    while (ss >> word) s_words.push_back(word);
    while (tt >> word) t_words.push_back(word);

    int i = 0, j = 0;
    while (i < s_words.size()) {
        if (j < t_words.size() && s_words[i] == t_words[j]) {
            j++;
        } else {
            result.push_back(s_words[i]);
        }
        i++;
    }

    return result;
}

```

The image shows a laptop screen displaying a coding problem titled "Question 24" on the left and its solution in C++20 on the right.

Question 24:

An office supply store offers an ink cartridge recycling program where customers can either earn money for each recycled cartridge or combine recycled cartridges with cash to buy special products known as "perk items." A customer participating in this program wants to maximize the number of perk items they can purchase. Given a specific number of cartridges and dollars, determine how many perk items the customer can acquire.

Example

cartridges = 10
dollars = 10
recycleReward = 2, the amount earned by recycling a single cartridge
perkCost = 2, the amount required for a customer to buy a single perk item combined with a recycled cartridge

The best thing the customer can do is to first recycle 3 cartridges, earning 6 dollars. After this, 7 cartridges and 16 dollars are left. The 7 cartridges can be combined with 14 dollars to purchase 7 perk items. This is the maximum number of perk items that can be bought.

Function Description

Complete the `maxPerkItems` function in the editor with the following parameter(s):

- `int cartridges`: the number of cartridges on-hand
- `int dollars`: the number of dollars on-hand
- `int recycleReward`: dollars earned per recycled cartridge
- `int perkCost`: dollar cost of a perk item, in addition to recycling a single cartridge

Returns

`int`: the maximum number of perk items that can be bought

Constraints

- $1 \leq \text{cartridges, dollars, recycleReward, perkCost} \leq 10^9$

Input Format For Custom Testing

Sample Case 0

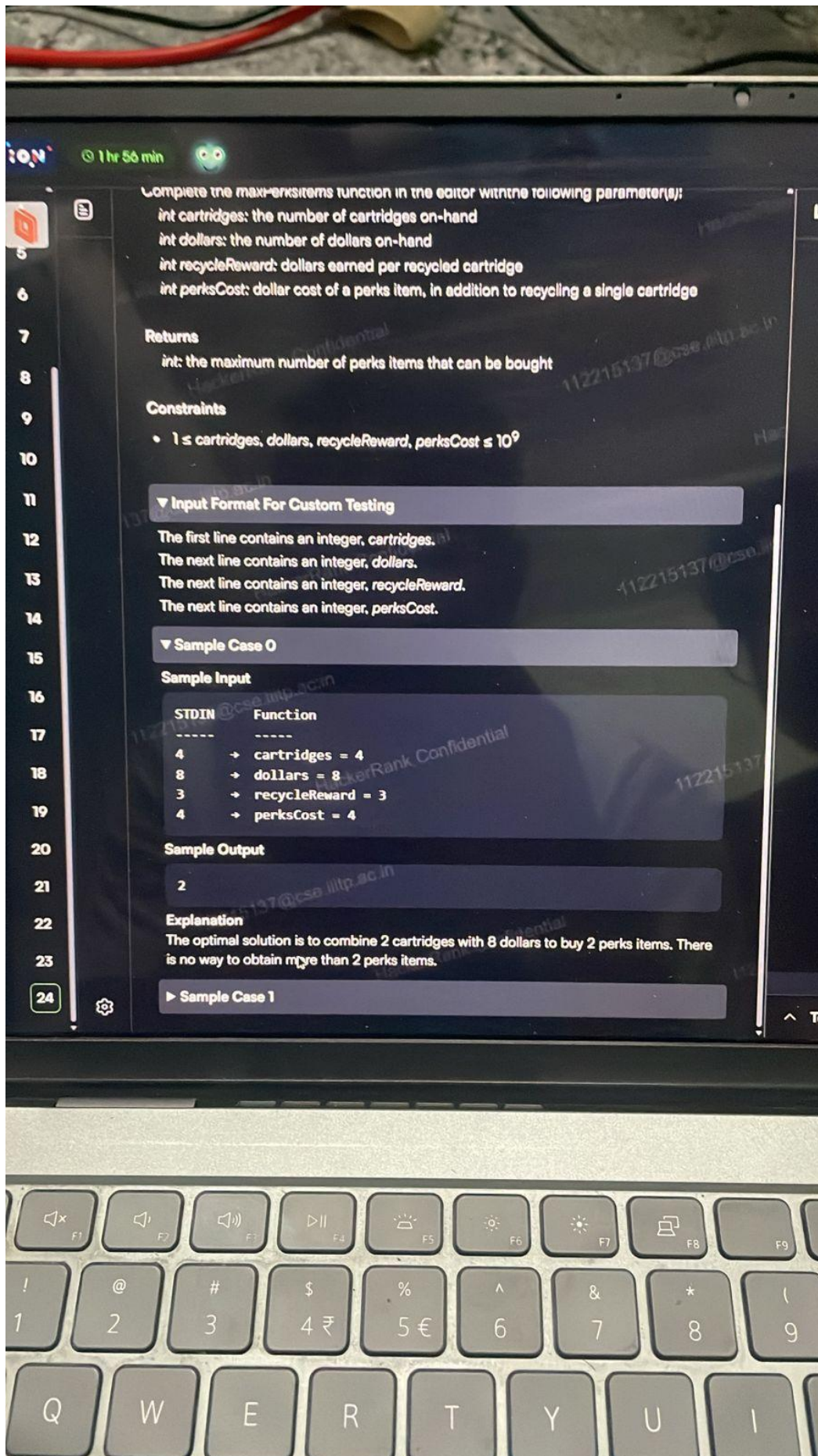
Solution (C++20):

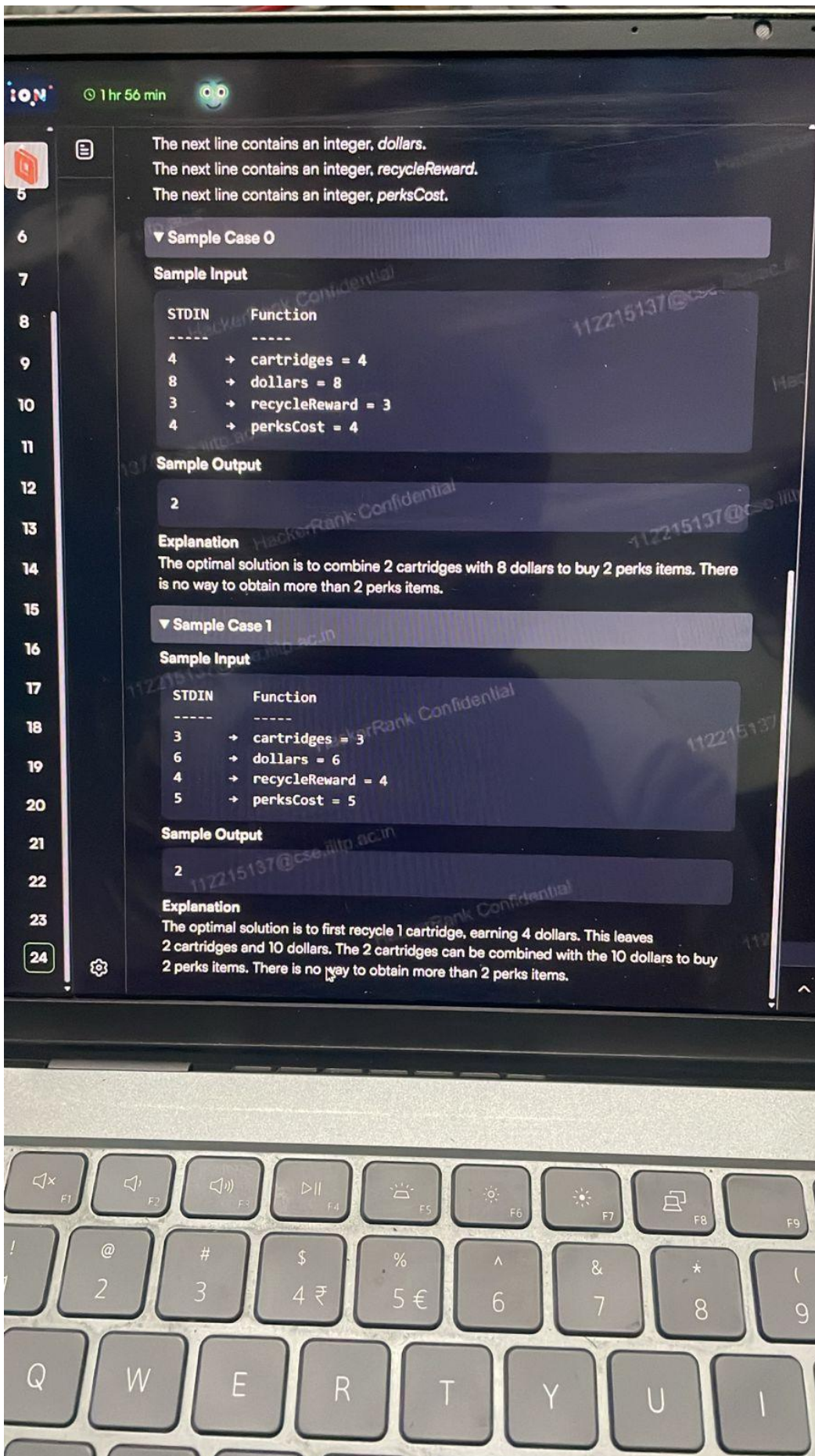
```

1 > #include <bits/stdc++.h>...
9
10
11 * Complete the 'maxPerkItems' function below.
12 *
13 * The function is expected to return an INTEGER.
14 * The function accepts following parameters:
15 * 1. INTEGER cartridges
16 * 2. INTEGER dollars
17 * 3. INTEGER recycleReward
18 * 4. INTEGER perkCost
19 */
20
21 int maxPerkItems(int cartridges, int dollars, int recycleReward,
22 int perkCost) {
23     // ...
24 }
25 > int main() ...

```

At the bottom of the code editor, it says "Ln 9, Col 1 • Autocomplete Spaces: 4 Mode: Normal". There is a "Run Code" button at the bottom right.






```

#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

int maxPerksItems(int cartridges, int dollars, int recycleReward, int perksCost) {
    ll low = 0, high = cartridges;
    ll ans = 0;

    while (low <= high) {
        ll mid = (low + high) / 2;

        ll neededCartridges = mid;
        ll neededMoney = mid * perksCost;

        ll remainingCartridges = cartridges - mid;

        // Money gained by recycling remaining cartridges
        ll recycledMoney = (remainingCartridges >= 0) ? remainingCartridges *
recycleReward : 0;

        ll totalMoney = dollars + recycledMoney;

        if (neededCartridges <= cartridges && neededMoney <= totalMoney) {
            ans = mid; // This number of perks is possible, try more
            low = mid + 1;
        } else {
            high = mid - 1; // Too much, reduce
        }
    }

    return (int)ans;
}

int main() {
    int cartridges, dollars, recycleReward, perksCost;
    cin >> cartridges >> dollars >> recycleReward >> perksCost;

    cout << maxPerksItems(cartridges, dollars, recycleReward, perksCost) << endl;
    return 0;
}

```